



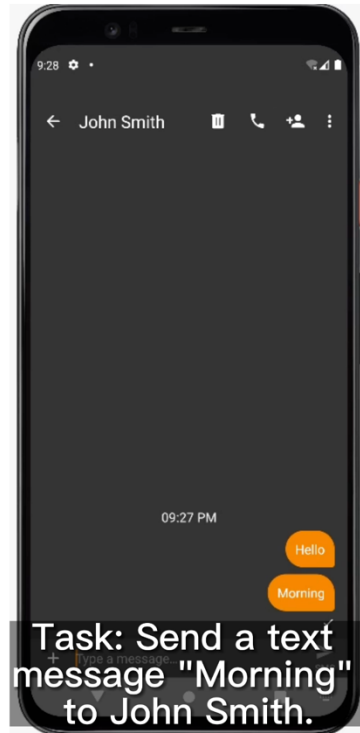
ShadowNPU: System and Algorithm Co-design for NPU-Centric On-Device LLM Inference

Wangsong Yin¹, Daliang Xu², Mengwei Xu², Gang Huang¹, Xuanzhe Liu¹

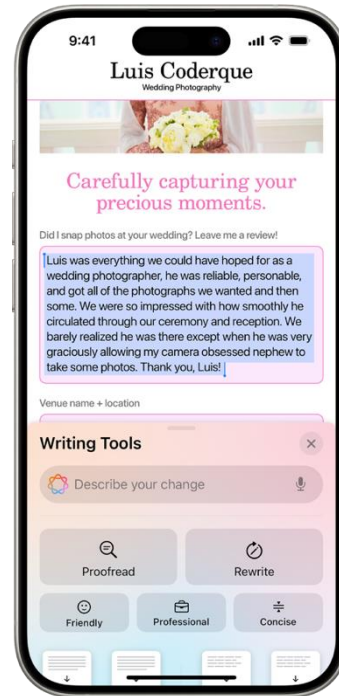
¹Peking University, ²Beijing University of Posts and Telecommunications

MobiSys 2026

On-Device Large Language Models



Automatic GUI Manipulation



Screen Comprehension

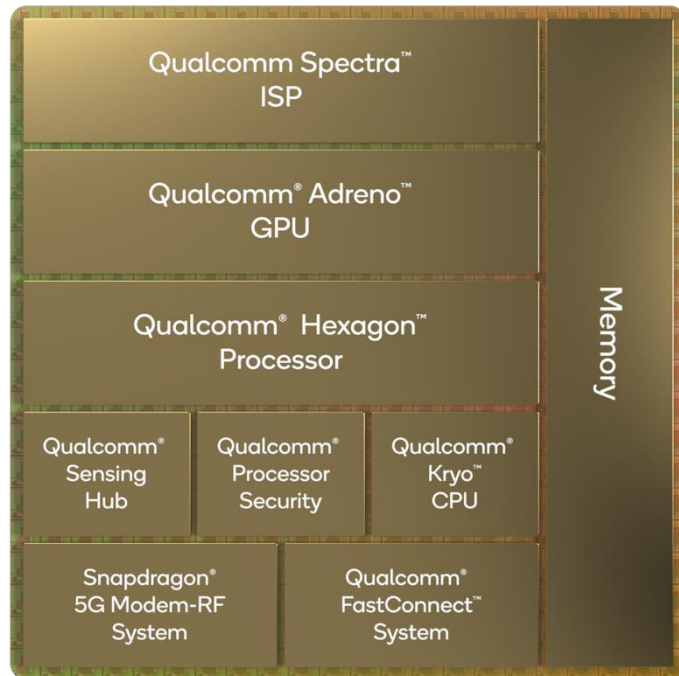


Voice Assistant

Privacy Sensitive!

Needs On-Device Deploying LLMs!

Mobile SoC

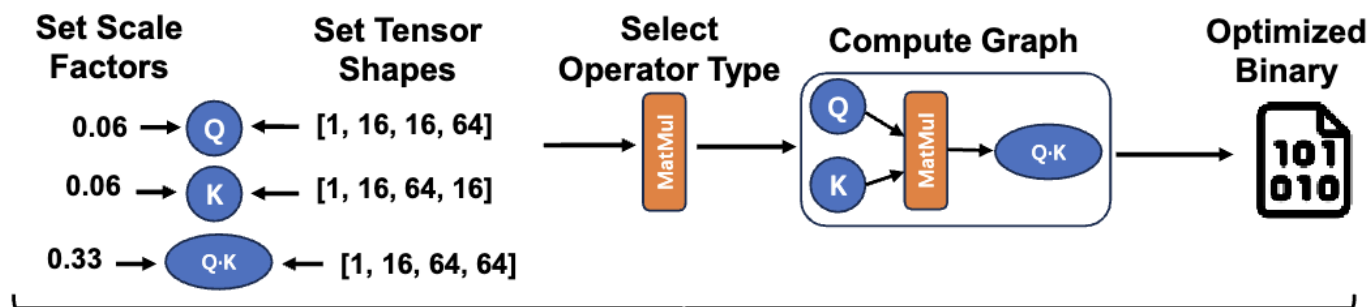


CPU/GPU: sparse FP compute
NPU: dense INT compute

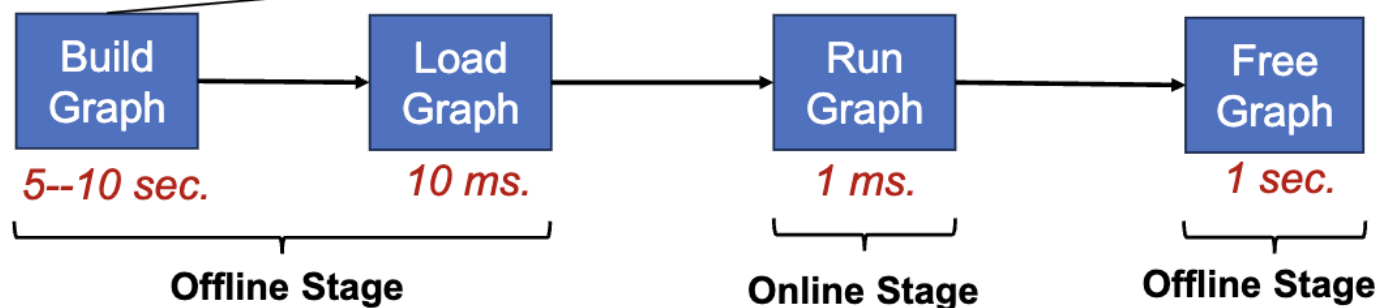
Vendor	SoC	NPU	INT Capability	Float Capability
Qualcomm	Snapdragon	Hexagon DSP	High	No
Qualcomm	Snapdragon	HTP	High	Low
Texas Instruments	TMS320F2812	C28x DSP	High	No
Nvidia	Orin	DLA	High	No
Tesla	FSD	FSD D1	High	No

The high-throughput integer compute ability

Mobile NPUs: No-Workaround Limitations



Static compute shape



Coarse-grained static quantization

Status Quo of On-Device Frameworks

Framework	llm.npu	HeteroLLM	llama.cpp	mlc-llm
Atten.	CPU	GPU	CPU	GPU
Non-Atten.	NPU	GPU/NPU	CPU	GPU

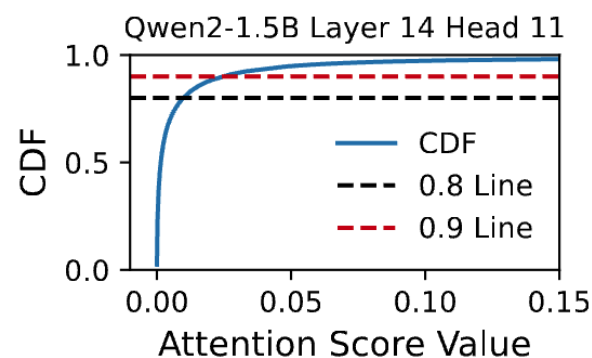
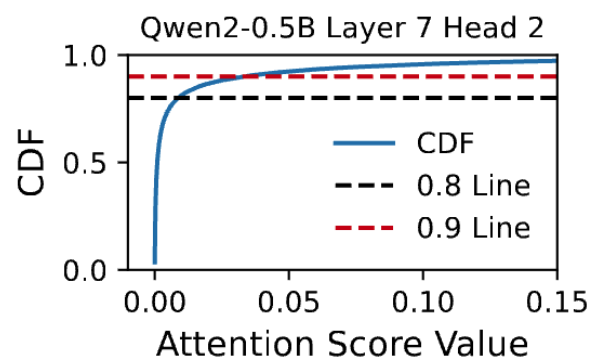
Dataset	PhoneLM -0.5B		PhoneLM -1.5B		Qwen2 -0.5B		Qwen2 -1.5B	
	C/G	N	C/G	N	C/G	N	C/G	N
ArxivSum [18]	14.7	0.0	11.9	0.0	10.7	9.4	8.5	9.1
DroidCall [61]	27.5	20.5	20.5	19.0	34.5	27.5	48.0	22.5
Octopus [17]	64.6	24.1	79.2	24.7	60.6	34.8	61.2	34.2

CPU/GPU fallback – Not NPU-centric!

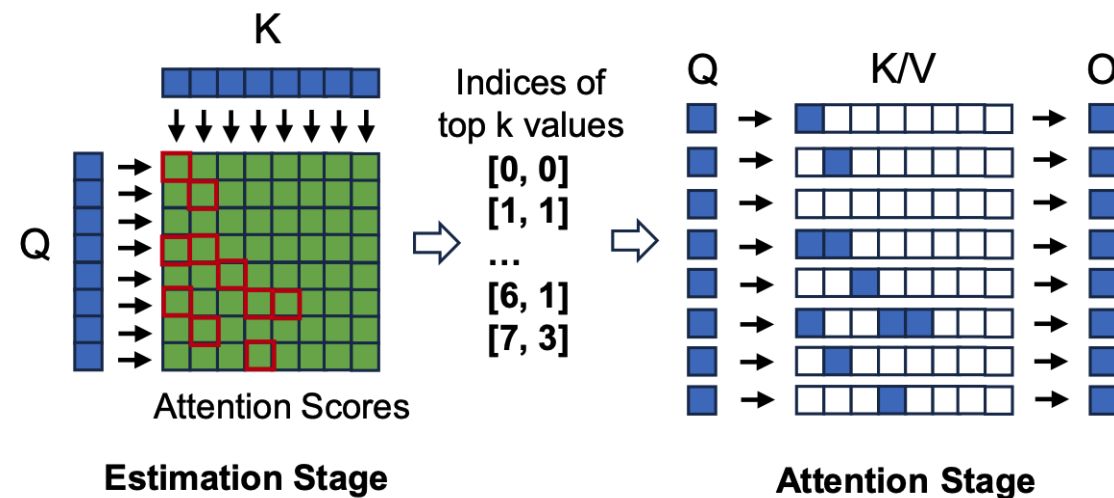
Mainly due to quantization error

Can we (almost) fully offload the LLM inference workload to dedicated NPUs, minimizing the burden of CPU/GPU?

The Opportunity: Sparse Attention

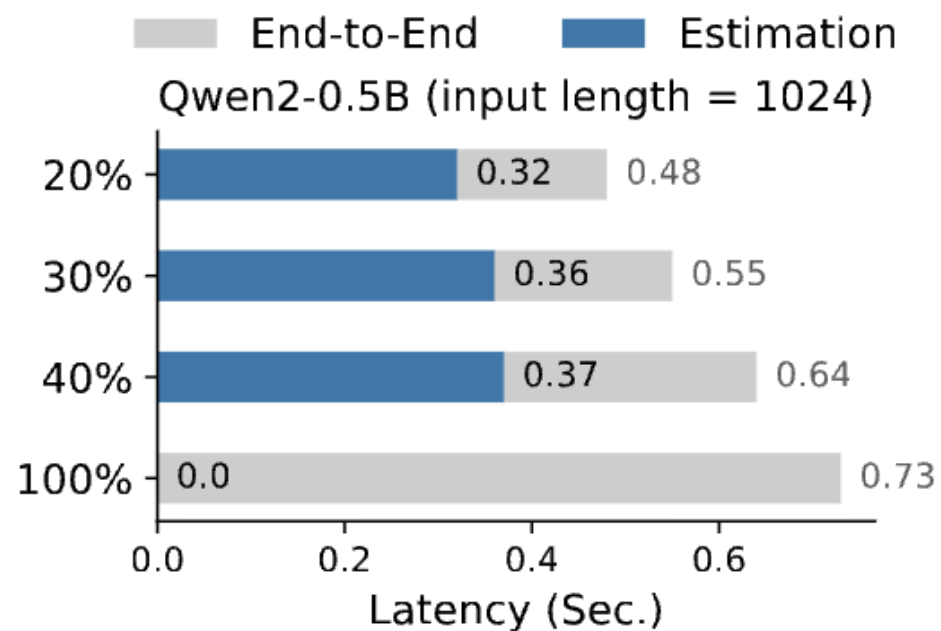


The attention can be highly sparse



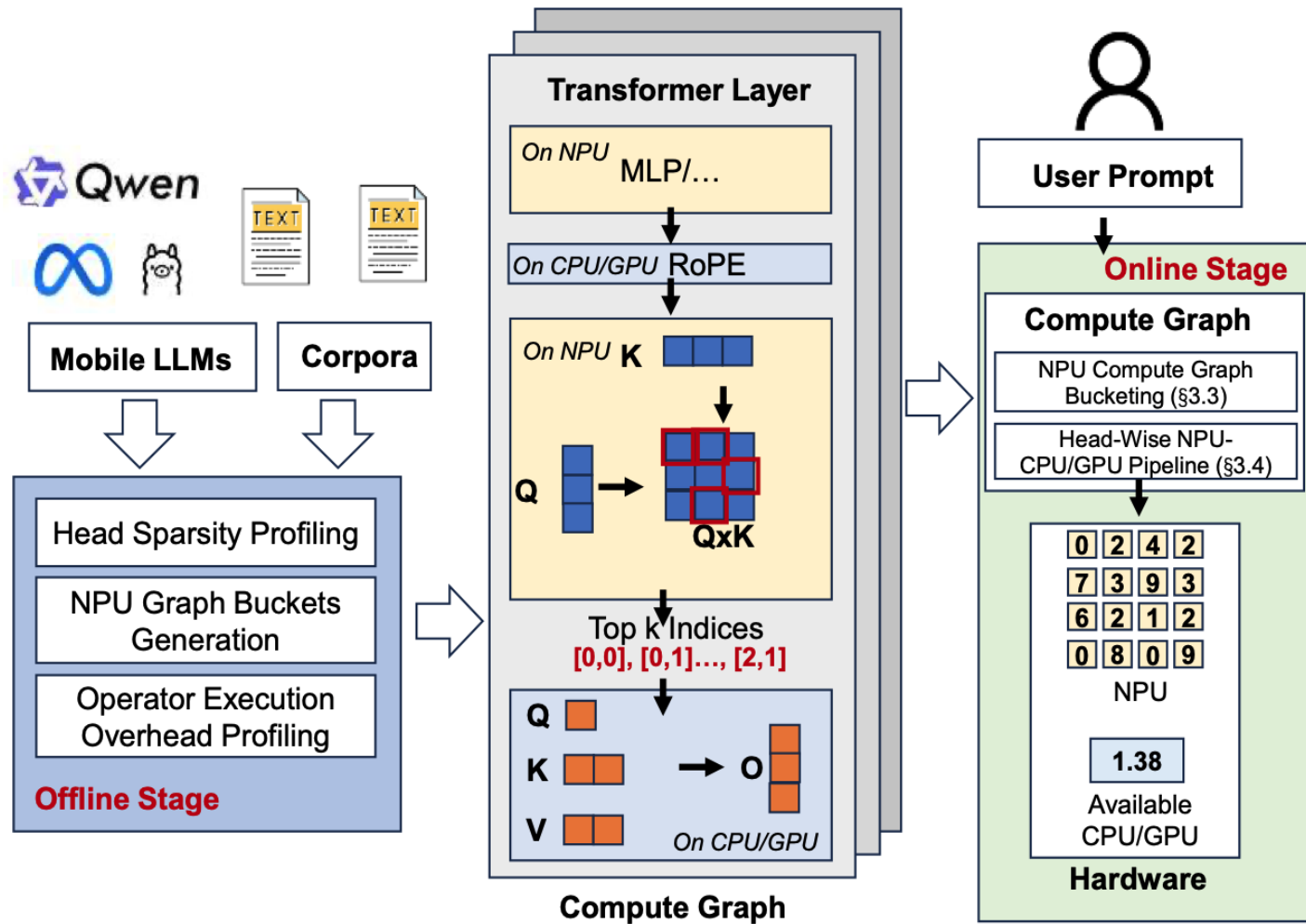
General workflow

The Challenge: The Estimation Stage



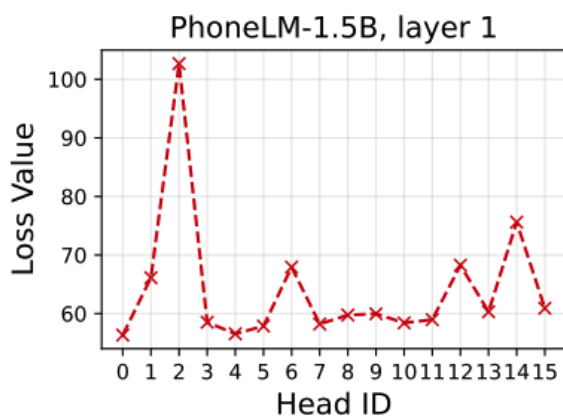
Hidden bottleneck: the estimation stage

Our System

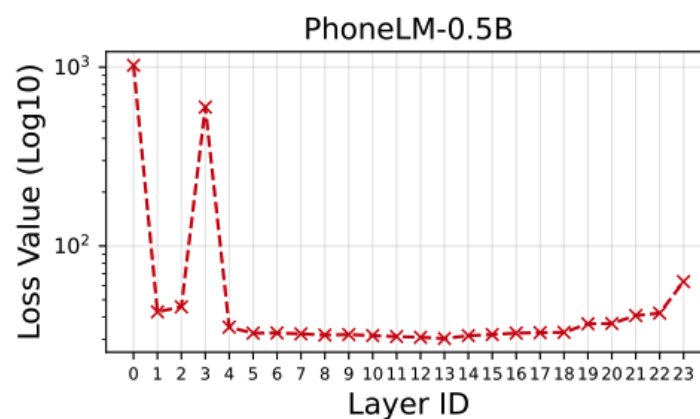


**NPU-centric
attention module!**

Head-Specific Sparse Ratio



(a) Loss on each head.



(b) Loss on each layer.

Head/layer importance

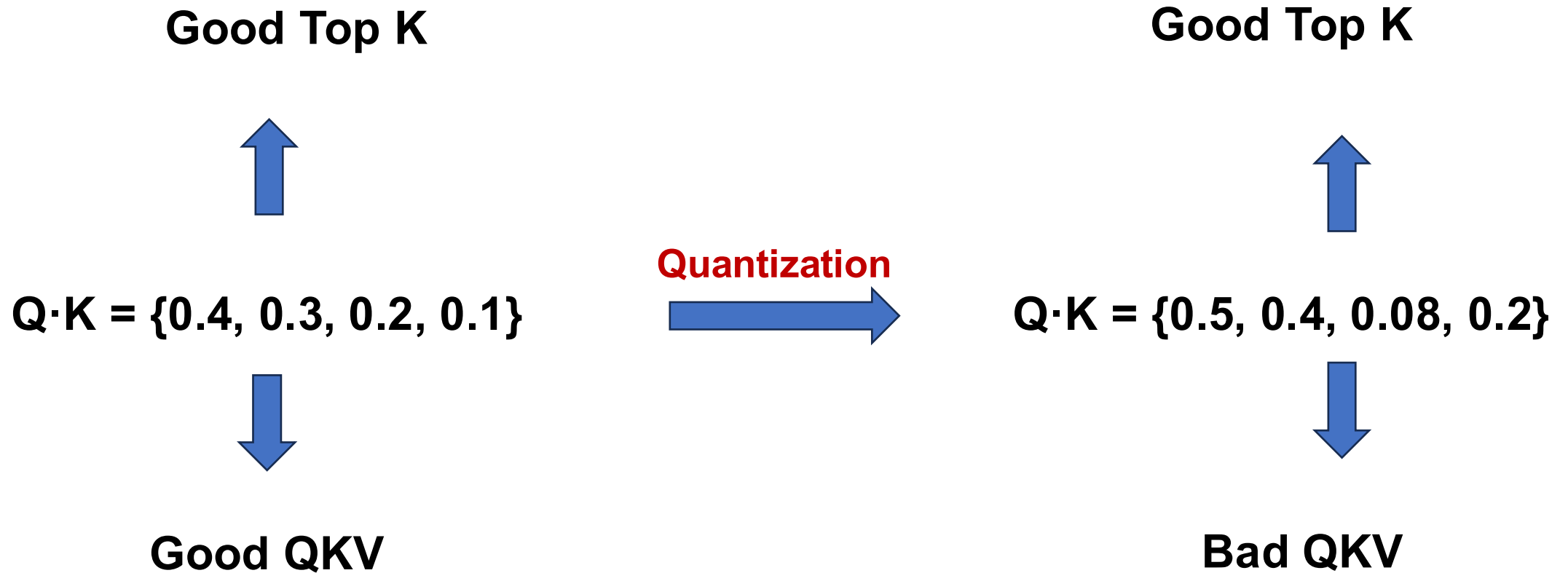
$$headImp_i = loss(head_i = 0, C) - loss(C),$$

$$layerImp_j = loss(layer_j = 0, C) - loss(C).$$

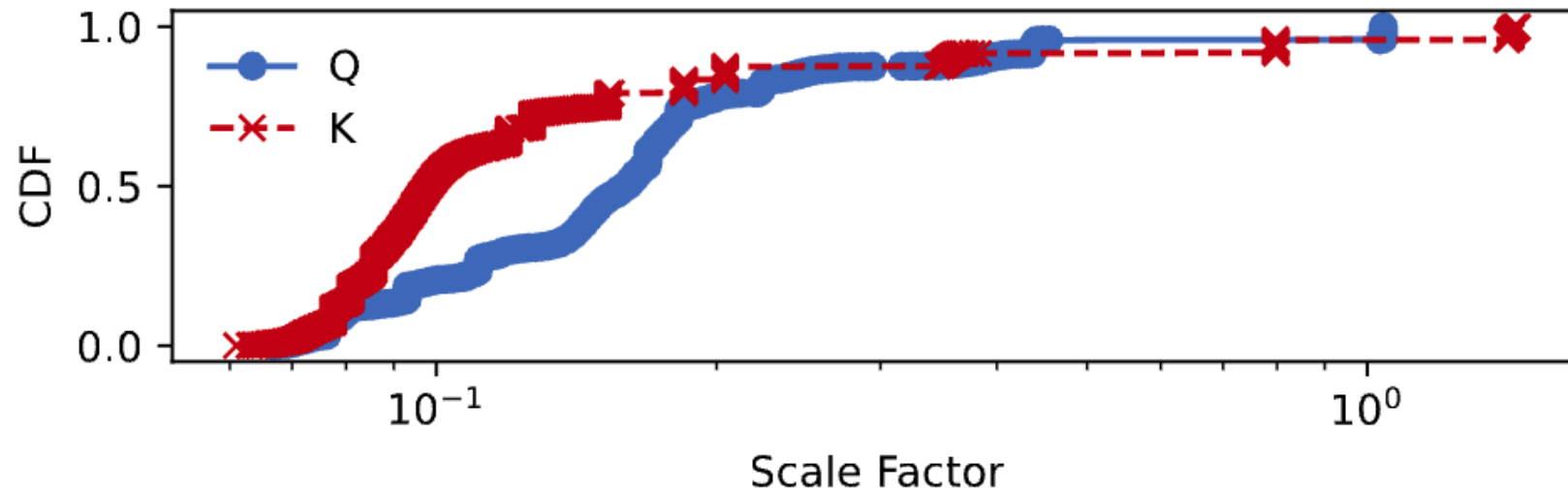
Head sparsity ratio

$$\frac{r \cdot N \cdot \text{clamp}(headImp_i \cdot layerImp_j)}{\text{sum}(\text{clamp}(headImp_i \cdot layerImp_j))}, 1 \leq i \leq N,$$

Running the Estimation on NPU

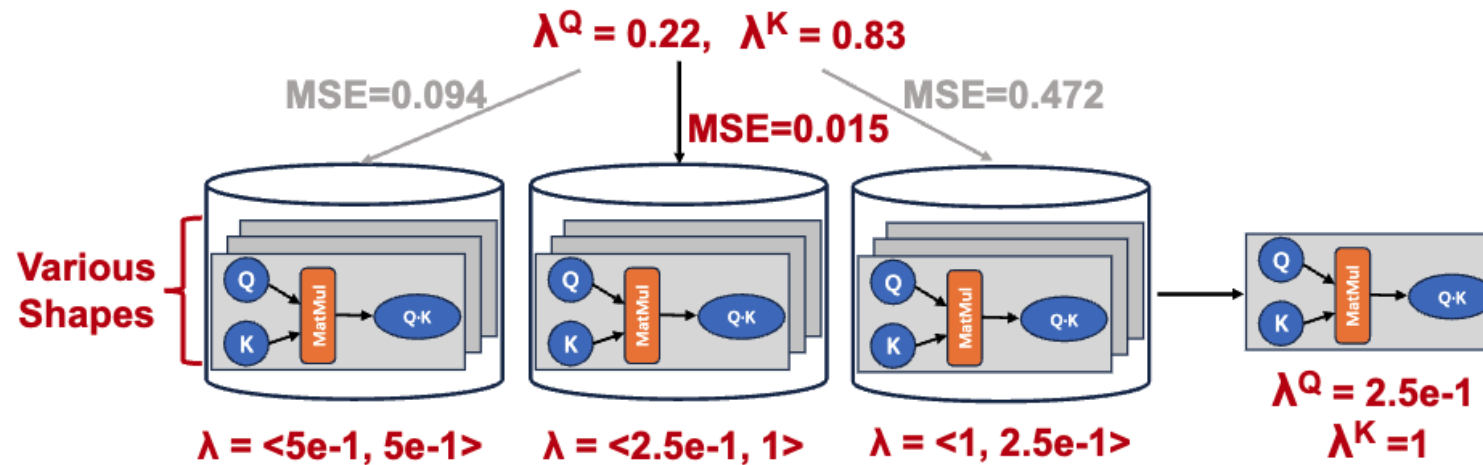


NPU Compute Graph Bucketing



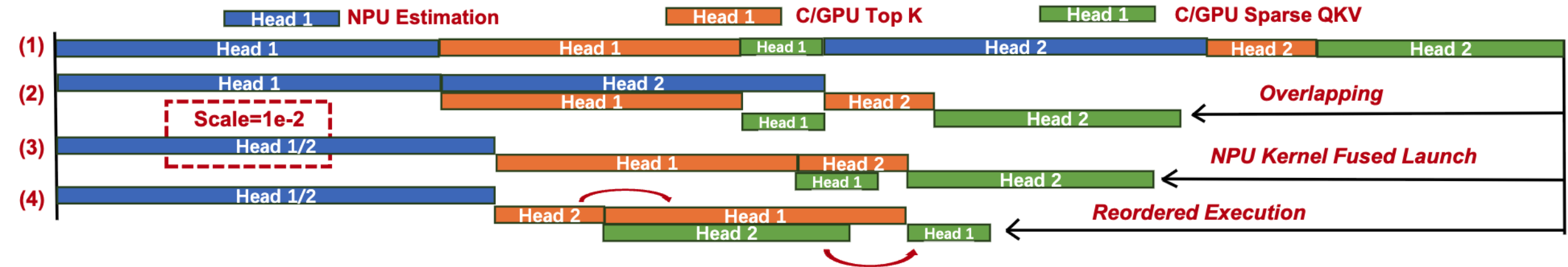
The scale factors are fluctuated

NPU Compute Graph Bucketing



Our solution: bucketing the graphs

Head-Wise NPU-CPU/GPU Pipeline



Overlapping the heads with several insights

Reducing the planning overhead by greedy search

Implementation

■ ***Models***

- PhoneLM-0.5B
- PhoneLM-1.5B
- Qwen2-0.5B
- Qwen2-1.5B

■ ***Datasets***

- ArxivSum
- DroidCall
- Octopus

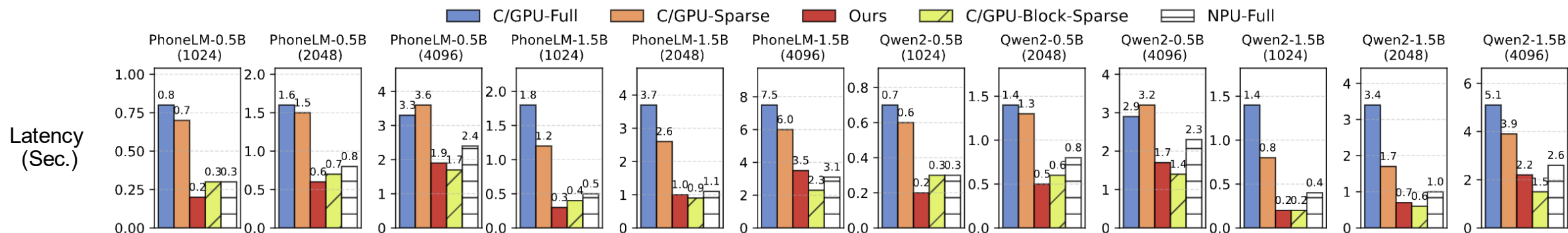
■ ***Devices***

- MI14 Smartphone
- Redmi K60 Champion Edition

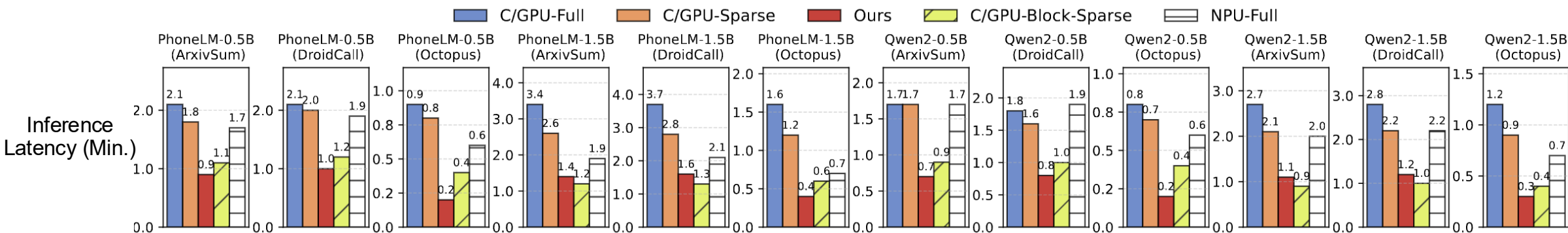
■ ***Baselines***

- Full attention on CPU/GPU (C/G-Full)
- Sparse attention on CPU/GPU (C/G-Sparse)
- Block sparse attention on CPU/GPU (C/G-Block-Sparse)
- Full attention on NPU (NPU-Full)

Evaluation

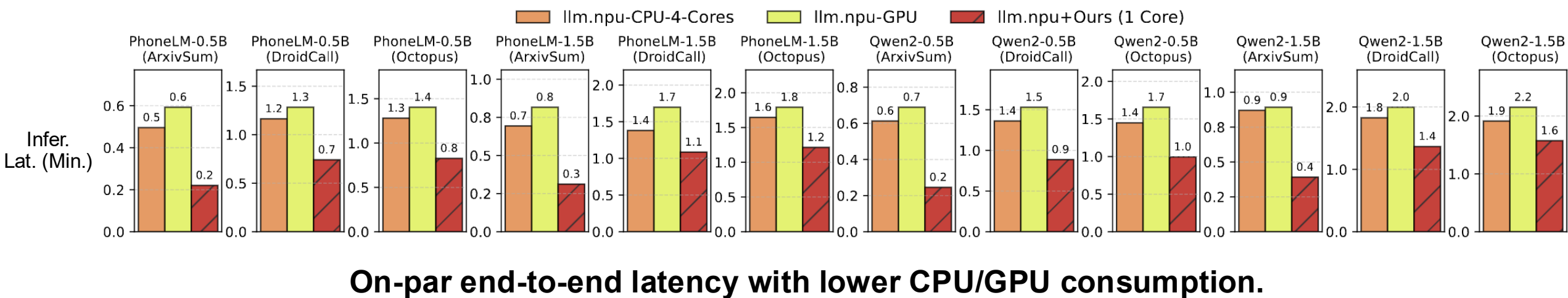


Lower kernel latency with limited CPU/GPU resource.



Lower end-to-end latency with limited CPU/GPU resource.

Evaluation



Evaluation

Our system is accurate. Compared to the lossless baseline C/G-Full, the accuracy loss is only 0.4 pp.

Model	Dataset	C/G-Full	C/G-Sparse	C/G-Block-Sparse	NPU-Full	Ours
PhoneLM -0.5B	ArxivSum	14.7	14.9	10.0	0.0	15.2
	DroidCall	27.5	24.0	25.5	20.5	25.5
	Octopus	64.6	71.3	62.9	24.1	64.0
PhoneLM -1.5B	ArxivSum	11.9	12.1	6.0	0.0	12.0
	DroidCall	20.5	23.5	28.5	19.0	25.5
	Octopus	79.2	75.8	45.5	24.7	75.8
Qwen2 -0.5B	ArxivSum	10.7	11.4	10.8	9.4	11.3
	DroidCall	34.5	34.5	26.0	27.5	37.5
	Octopus	60.6	42.1	37.6	34.8	61.2
Qwen2 -1.5B	ArxivSum	8.5	2.0	4.2	9.1	8.4
	DroidCall	48.0	19.0	15.5	22.5	44.5
	Octopus	61.2	21.9	32.0	34.2	56.1
Average		36.8	29.4	25.4	18.8	36.4

Take Aways

- We present ShadowNPU, an **on-device sparse attention kernel**.
- It leverages the characteristics of **heterogeneous processors on mobile SoC**.
- ShadowNPU delivers **SoTA performance** of attention.